

APPLE-DSE: Asynchronous Parallel Pareto Set Learning for Microarchitecture Design Space Exploration

Xuyang Zhao¹, Tianning Gao², Zheng Wu¹, Zhaori Bi¹, *Member, IEEE*, Changhao Yan¹, *Member, IEEE*, Fan Yang¹, *Member, IEEE*, Sheng-Guo Wang¹, *Life Senior Member, IEEE*, Dian Zhou, *Senior Member, IEEE*, and Xuan Zeng¹, *Senior Member, IEEE*

Abstract—The synthesizable and parameterizable RISC-V microarchitecture, combined with multiobjective optimization-based design space exploration (DSE), facilitates agile adaptation to various microprocessor designs for customized applications. However, to enhance design quality, DSE must consider both architecture parameters and EDA tool parameters, resulting in exponentially increased optimization complexity with the dimensionality of parameters. Exhaustively exploring the whole design space is impossible. Additionally, due to the time-consuming nature of microprocessor simulation, minimizing the number of simulations is imperative. Addressing these challenges, we propose asynchronous parallel Pareto set learning for microarchitecture DSE (APPLE-DSE). APPLE-DSE utilizes the Pareto set learning (PSL) technique to obtain an approximate Pareto front with a “light-weight” evaluation. PSL captures the structural characteristics of the Pareto set (PS) guided by the surrogate models, enabling it to explore any tradeoff area in the approximate PS. Employing the probabilistic reparameterization (PR) technique, APPLE-DSE adapts PSL to handle discrete variables. Furthermore, APPLE-DSE incorporates a simulation time-aware asynchronous parallel scheduling strategy to further enhance optimization efficiency. Experimental results show that APPLE-DSE achieves a maximum improvement of 16.81% in hypervolume within the same time budget and a 127.73× speedup in algorithm run time per iteration compared to state-of-the-art methods.

Index Terms—Asynchronous parallel, design space exploration (DSE), Pareto set learning (PSL), RISC-V microarchitecture.

I. INTRODUCTION

MICROPROCESSOR design has witnessed a paradigm shift toward customization, driven by diverse application domains, including mobile devices, automotive systems, and medical equipment [1]. The varying application requirements necessitate specific Quality of Results (QoRs) metrics, notably performance, power, and area (PPA). Simultaneously, the increasing adoption of multiprocessor system-on-chips (MPSoC) [2] underscores the need for microprocessors to align seamlessly with overall system design objectives. System-level optimization often entails iterative fine-tuning, which may involve adjusting the PPA objectives of individual microprocessors.

Conventional optimization methods often focus solely on a single QoR metric, while the need to consider tradeoffs among multiple QoR metrics is neglected. For instance, in Fig. 1, Design 1 exhibits lower-power consumption but higher delay, while Design 2 demonstrates lower delay but higher-power consumption. Absent specific application requirements, discerning the superior design is challenging. Addressing the need to balance tradeoffs among multiple QoR metrics, designers typically employ two strategies. First, they may assign weights (preferences) to each metric and optimize a weighted Figure of Merit (FoM). Alternatively, designers may opt to optimize a single QoR metric while imposing constraints on the others. However, both approaches necessitate rerunning the optimization process whenever there are changes in the QoR specifications.

The multiobjective optimization (MOO) addresses the constraints inherent in single-objective optimization (SOO). Unlike SOO, which provides a singular optimal solution, MOO generates a collection of Pareto optimal solutions. For example, as shown in Fig. 1, both Design 1 and Design 2 are considered Pareto optimal within MOO. The ensemble of Pareto optimal solutions constitutes the Pareto set (PS), while their corresponding objective values construct the Pareto front (PF). A solution that lags behind another design across all objectives is deemed “dominated,” as depicted by the blue point in Fig. 1. The hypervolume (HV) metric quantifies the PF’s quality, with a larger HV indicating a superior PF.

Received 11 July 2024; revised 19 October 2024 and 16 December 2024; accepted 22 December 2024. Date of publication 26 December 2024; date of current version 20 June 2025. This work was supported in part by the National Key R&D Program of China under Grant 2020YFA0711900 and Grant 2020YFA0711901; and in part by the National Natural Science Foundation of China (NSFC) Research Projects under Grant 62474050, Grant 62304052, Grant 62141407, Grant 92473207, Grant 62474051, and Grant 92373207. This article was recommended by Associate Editor K. Olcoz. (Corresponding authors: Zhaori Bi; Xuan Zeng.)

Xuyang Zhao, Zheng Wu, Zhaori Bi, Changhao Yan, Fan Yang, and Xuan Zeng are with the School of Microelectronics, State Key Laboratory of Integrated Chips and Systems, Fudan University, Shanghai 200433, China (e-mail: zhaori_bi@fudan.edu.cn; xzeng@fudan.edu.cn).

Tianning Gao is with the Department of Electrical Engineering, University of Texas at Dallas, Dallas, TX 75080 USA.

Sheng-Guo Wang is with the School of Microelectronics, State Key Laboratory of Integrated Chips and Systems, Fudan University, Shanghai 200433, China, and also with the College of Engineering, The University of North Carolina at Charlotte, Charlotte, NC 28223-0001 USA.

Dian Zhou is with the School of Microelectronics, State Key Laboratory of Integrated Chips and Systems, Fudan University, Shanghai 200433, China, and also with the Department of Electrical Engineering, University of Texas at Dallas, Dallas, TX 75080 USA.

Digital Object Identifier 10.1109/TCAD.2024.3522880

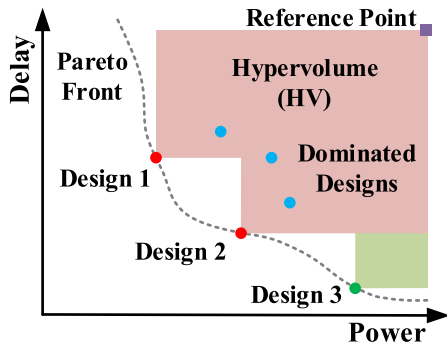


Fig. 1. Illustration of MOO. In MOO, both Design 1 and Design 2 are Pareto optimal. The set comprising these designs is the PS and the corresponding objectives constitute the PF. If a solution is inferior to another design in all objectives, it is considered dominated, as shown by the blue point. The HV enclosed by the PF points and a reference point is used to evaluate the quality of the PF. The larger HV means the better-MOO result. The HV of the PF, including only Design 1 and Design 2, is represented by the red area. If Design 3 is added to the PF, the HV increases by the green area.

As demonstrated in Fig. 1, with a reference point, the HV of the PF encompassing only Design 1 and Design 2 is delineated by the red area. Upon the addition of Design 3 (green point) to the PF, the HV expands by the green area. A more favorable PF signifies a broader spectrum of candidate designs. Hence, MOO adeptly catalogs all potential optimal designs across diverse application scenarios without necessitating a reexploration of the design space. Designers can readily reassess the most suitable design in instances of altered application requirements.

Recently, the RISC-V architecture [3] has garnered significant attention for its open-source licensing and inherent flexibility. For the RISC-V system-on-chip (SoC), the Berkeley architecture research (BAR) Group has developed the Chipyard [4] framework. It utilizes Chisel [5] hardware construction language to generate the synthesizable and parameterizable processor cores (such as BOOM [6], [7] and Rocket [8]) in the SoC. This framework allows for rapid customization of various design parameters and the generation of corresponding register transfer level (RTL) netlists, facilitating agile adaptation to diverse design requirements.

To enhance the optimization quality of microarchitecture design space exploration (DSE) problems, it is crucial to integrate electronic design automation (EDA) tool parameters into the MOO process. Existing methods for evaluating microprocessors often rely on simulators like GEM5 [9]. Discrepancies between simulation results and actual design outcomes necessitate manual adjustments later in the design process. To achieve PPA results that closely resemble the actual hardware, the Chipyard framework leverages the HAMMER [10] framework for automated very-large-scale integration (VLSI) flow. This framework utilizes EDA tools for RTL simulation, logic synthesis, power analysis, and other tasks, ensuring the accuracy of PPA results. Chipyard supports automated generation from Chisel to layout. However, integrating EDA tool parameters into DSE introduces higher dimensionality, with an increased number of design parameters that expand the design space. For instance, integrating logic synthesis tool parameters into DSE can result in over 50

design parameters and a design space exceeding 10^{30} in size. In the SoC design process, despite designers' professional knowledge, the growing complexity of SoCs and the opaque optimization methods in EDA tools can lead to optimal design parameter configurations appearing in unexpected locations. So, the DSE methods with the VLSI flow can empower designers to broaden their perspectives and discover superior designs.

Moreover, the time required to complete VLSI flows is significant. For instance, in the case of the BOOM microprocessor, completing RTL generation, simulation, and logic synthesis can take up to 6 h. Consequently, DSE cannot exhaustively explore the entire design space within a limited time budget. Commonly used MOO methods, such as NSGA-II [11] and MOEA/D [12], based on evolutionary algorithms, are not suitable for microarchitecture DSE problems due to their extensive simulation requirements.

Recently, model-based methods, such as Bayesian optimization (BO), have emerged as promising approaches for addressing DSE challenges by leveraging surrogate models to minimize the number of simulations required. Liu and Carloni [13] propose an iterative-refinement DSE method based on random-forest (RF) learning model and transductive experimental design (TED). This method can effectively find an approximate PS for high-level synthesis (HLS) with a limited number of simulation points. BOOM-Explorer [14] proposes a microarchitecture-aware initialization algorithm based on active learning MicroAL to get the characteristics of the BOOM microarchitecture, and utilizes the Gaussian process (GP) with deep kernel learning (DKL-GP) to model the PPA results of designs. Since MicroAL is an enhancement of TED tailored for BOOM microarchitecture DSE, BOOM-Explorer demonstrates effective exploration of the BOOM design space. Sherlock [15] integrates a model selection process based on the multiarmed bandit (MAB) to choose the model that helps find the optimal solution faster, so that it can adaptively solve the DSE problems. GRL-DSE [16] leverages a graph neural network (GNN) to represent the features of the RISC-V microarchitecture and employs the expected HV improvement (EHVI) acquisition function to obtain new simulation points, further enhancing the optimized HV. It also utilizes the ensemble learning to select the suitable surrogate model. However, the above methods [13], [14], [15], [16] necessitate the evaluation of the entire design space using surrogate models to determine the next data point for simulation, even when only a subset of design points requires simulation. These approaches are impractical for design spaces with 10^{30} candidates.

In the mathematical community, various methods [17], [18], [19], [20] have been proposed to address MOO problems characterized by high-dimensionality and vast parameter spaces. MORBO [17] employs a trust region approach to concentrate the optimization scope on local areas, thereby avoiding over-exploration in the expansive design space. On the other hand, REMBO [18] utilizes a random embedding matrix to map the high-dimensional design space to a low-dimensional one, effectively reducing the complexity associated with high-dimensional problems. However, these

methods primarily focus on continuous variable optimization problems. To address discrete variable optimization problems, BODi [19] utilizes dictionary embedding to map the discrete high-dimensional space to a continuous low-dimensional one, enabling it to handle discrete variables. Recently, REMOTune [20] integrates MORBO [17] and REMBO [18] to manage DSE with VLSI flows. By concentrating on the locally mapped low-dimensional design space, REMOTune achieves promising results. However, due to its local perspective, REMOTune may overlook the correlation between Pareto optimal points, potentially leading to suboptimum solutions.

To alleviate the challenges posed by time-consuming simulations in DSE problems, parallel simulation techniques have been introduced. However, the varying simulation times for different design parameter configurations can lead to inefficient utilization of computational resources. Synchronous parallel simulation methods, exemplified by REMOTune [20], necessitate the completion of all simulations in the previous iteration before commencing simulations in the next iteration, resulting in idle time gaps. To address these inefficiencies, we propose asynchronous parallel Pareto set learning (PSL) for microarchitecture DSE (APPLE-DSE). APPLE-DSE integrates the PSL technique, which leverages surrogate models for “light-weight” evaluation to approximate the PF efficiently. PSL captures the structural characteristics of the PS guided by surrogate models, enabling exploration of tradeoff areas. Incorporating the probabilistic reparameterization (PR) technique, APPLE-DSE extends PSL to handle discrete variables. Additionally, APPLE-DSE adopts a simulation time-aware asynchronous parallel scheduling strategy to further enhance optimization efficiency. Our contributions are summarized as follows.

- 1) We propose a PSL model-based DSE method for microarchitecture, effective for high-dimensional MOO problems with vast design spaces and time-consuming simulations. In the training phase, the PSL model learns the structural characteristics of the sampled PS. To alleviate the computational burden associated with costly design evaluations, we leverage surrogate models for estimating design quality in an online updating mode, significantly reducing simulation costs. In the prediction phase, the trained PSL model directly generates candidate PS points based on specified preferences, facilitating the exploration of tradeoffs in the PPA space. By bypassing the necessity of optimizing acquisition functions or evaluating the entire design space, the proposed method is liberated from constraints imposed by the size or dimensionality of the design space.
- 2) We extend the PSL model to discrete DSE problems by the PR technique. The PSL model requires gradients for training, but microarchitecture DSE is a discrete optimization problem. Therefore, the original PSL model cannot be directly applied to DSE problems. The PR technique allows APPLE-DSE to treat design parameters as differentiable entities. This enables gradient-based updates to the PSL model. The discrete design parameters can then be sampled from the probability distribution of the PR.

- 3) We propose a simulation time-aware asynchronous parallel scheduling strategy to minimize resource idling and optimize efficiency within a constrained time budget. To avoid duplicate point selection, we generate candidate points based on diverse preferences. These points are then sorted using the HV improvement (HVI) metric, penalizing designs already selected for simulation. Additionally, we model simulation time to prioritize points that can be completed within the remaining time budget.

The experimental results demonstrate that APPLE-DSE outperforms the state-of-the-art methods, achieving 12.23% ~ 16.81% improvement of HV within the same time budget and $3.12\times \sim 127.73\times$ speed-up of algorithm run time per iteration. In terms of PPA metrics, APPLE-DSE achieves improvements of up to 43.58% in performance, 38.36% in power consumption, and 13.81% in area. Compared with the official designs of RISC-V BOOM and Rocket microarchitecture, APPLE-DSE can obtain 24.70% ~ 28.93% improvement of power with higher performance and smaller area.

The remainder of this article is structured as follows. Section II introduces the problem formulation and the GP regression (GPR) model. Section III details the APPLE-DSE algorithm. Section IV presents the experimental results and compares the performances of our proposed algorithm with state-of-the-art algorithms. Section V concludes this article.

II. BACKGROUND

A. Problem Formulation

The microarchitecture DSE problem can be formulated as a MOO problem, as shown in

$$\text{minimize } f_1(\mathbf{x}), \dots, f_M(\mathbf{x}) \quad (1)$$

where $\mathbf{x}$ denotes input parameters and $f_i(\cdot)$ represents objective functions. In this work, $\mathbf{x} = \{x_1, x_2, \dots, x_N\}$ includes the design parameters, such as *FetchWidth*, *numFetchBufferEntries*, and *DecodeWidth*. $f_i(\cdot)$ includes power consumption, area, and the average cycle per instruction (CPI) derived from different benchmark simulations, such as *dhrystone*, *mt-matmul*, and *qsort*.

In DSE problems, determining the superiority of one design over another is challenging. A design with a lower CPI may have higher-power consumption and a larger area, while a design with lower-power consumption and a smaller area may have a higher CPI. To evaluate the quality of MOO, the concept of *domination* is introduced. For arbitrary data points $\mathbf{a}$ and $\mathbf{b}$, $\mathbf{a} \prec \mathbf{b}$ ($\mathbf{a}$ dominates $\mathbf{b}$) if

$$\begin{aligned} & \forall i \in \{1, \dots, M\} \quad f_i(\mathbf{a}) \leq f_i(\mathbf{b}) \\ & \text{and } \exists i \in \{1, \dots, M\} \quad f_i(\mathbf{a}) < f_i(\mathbf{b}). \end{aligned} \quad (2)$$

The MOO yields a set of nondominated solutions, known as the PS, as defined in

$$\text{PS} = \{\mathbf{x} \in X \mid \nexists \mathbf{x}' \in X, \mathbf{x}' \prec \mathbf{x}\} \quad (3)$$

where X denotes input space. The objective space corresponding to the PS is commonly referred to as the PF in

$$PF = \{f_i(\mathbf{x}) | \mathbf{x} \in PS, i = 1, \dots, M\}. \quad (4)$$

In MOO, HV is a widely used performance indicator to assess the quality of the PF. As Fig. 1 illustrates, given a reference point $\mathbf{r}$ and a finite PF PF, the HV is the M -dimensional Lebesgue measure $\mathcal{M}$ of the space dominated by PF, as defined in

$$HV(PF, \mathbf{r}) = \mathcal{M}\left(\bigcup_{j=1}^{|PF|} [\mathbf{r}, \mathbf{y}_j]\right) \quad (5)$$

where $[\mathbf{r}, \mathbf{y}_j]$ denotes the hyper-rectangle bounded by $\mathbf{r}$ and $\mathbf{y}_j$, with $\{\mathbf{y}_j\}_{j=1}^{|PF|}$ comprising the PF. A larger HV value indicates a better PF, which corresponds to a better-MOO result.

B. Surrogate Model

GPR model [21], [22] is a commonly used surrogate model in BO. It can combine prior knowledge and new simulation points to estimate a posterior distribution with predictive mean and uncertainty.

A GP is a collection of random variables, any finite number of which have a joint Gaussian distribution. A GP is fully characterized by its mean function $m(\cdot)$ and its kernel or covariance function $k(\cdot, \cdot)$. If f follows a GP, the GPR model can provide posterior distribution for an arbitrary location $\mathbf{x}$ as follows [22]:

$$\begin{cases} \mu(\mathbf{x}) = m(\mathbf{x}) + k(\mathbf{x}, X)(K + \sigma_n^2 I)^{-1}(\mathbf{y} - m(X)) \\ \sigma^2(\mathbf{x}) = k(\mathbf{x}, \mathbf{x}) - k(\mathbf{x}, X)(K + \sigma_n^2 I)^{-1}k(X, \mathbf{x}) \end{cases} \quad (6)$$

where $\mu(\mathbf{x})$ is the predictive mean, $\sigma(\mathbf{x})$ represents the standard deviation, $k(\mathbf{x}, X) = \{k(\mathbf{x}, \mathbf{x}_i) | i = 1, \dots, N\}$, $k(X, \mathbf{x}) = k(\mathbf{x}, X)^T$, and $K = k(X, X)$. If there is no prior information about $f(\mathbf{x})$, $m(\mathbf{x})$ is always set to zero.

In this work, we set $m(\mathbf{x}) = 0$ and the kernel function as the Matern52 kernel [22], as defined in

$$k(\mathbf{x}_i, \mathbf{x}_j) = \frac{1}{\Gamma(\nu)2^{\nu-1}} \left(\frac{\sqrt{2\nu}}{l} d(\mathbf{x}_i, \mathbf{x}_j) \right)^\nu K_\nu \left(\frac{\sqrt{2\nu}}{l} d(\mathbf{x}_i, \mathbf{x}_j) \right) \quad (7)$$

where $\nu = 5/2$, $d(\cdot, \cdot)$ is the Euclidean distance, $K_\nu(\cdot)$ is a modified Bessel function, and $\Gamma(\cdot)$ is the Gamma function.

III. PROPOSED METHOD

A. Overview of APPLE-DSE Framework

The APPLE-DSE framework, depicted in Fig. 2, comprises distinct phases delineated by color coding: red for training, blue for prediction, and green for simulation.

1) *Training Phase*: APPLE-DSE initiates by constructing GPR models based on existing simulation data points. These existing data points may consist of designs known to the designers or random designs sampled by MicroAL [14] or TED [13]. The GPR models serve as surrogate representations of QoR metrics, effectively reducing the need for extensive simulations. Subsequently, a PR-based PSL model is trained utilizing the GPR models.

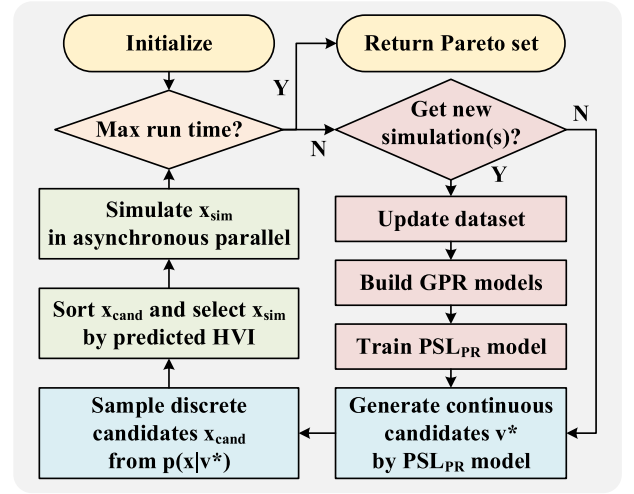


Fig. 2. Framework of APPLE-DSE method. The red part denotes the training phase, the blue part denotes the prediction phase, and the green part denotes the simulation phase.

2) *Prediction Phase*: Leveraging the trained PSL model, APPLE-DSE generates continuous candidate vectors $\mathbf{v}^*$ tailored to various preferences regarding PPA metrics. By sidestepping inefficient exploration within the expansive design space and obviating the need for costly acquisition function optimization, this phase ensures efficient candidate selection. To obtain discrete candidate vectors $\mathbf{x}_{\text{cand}}$ suitable for microarchitecture DSE, APPLE-DSE employs the PR technique to sample $\mathbf{x}_{\text{cand}}$ from the probability distribution $p(\mathbf{x}|\mathbf{v}^*)$.

3) *Simulation Phase*: APPLE-DSE adopts an asynchronous parallel scheduling strategy to enhance the optimization efficiency. The predicted HVI metric is utilized to rank all candidate points. The candidate point exhibiting the highest HVI is then designated for asynchronous parallel simulation.

APPLE-DSE seamlessly progresses to the subsequent optimization step as soon as a worker becomes available, without awaiting the return of all simulation results. Upon the availability of new simulation outcomes, APPLE-DSE reverts to the training phase. Otherwise, it proceeds with the prediction phase utilizing the preceding PSL model. Upon reaching the maximum runtime budget, APPLE-DSE returns the resulting PS.

B. Pareto Set Learning

Current model-based DSE methods alleviate the need for time-consuming real simulations but struggle with efficiently identifying the next candidate point. Approaches like BOOM-Explorer [14], Sherlock [15], and GRL-DSE [16] require traversing the entire design space to compute model prediction values for selecting optimal candidates, rendering them impractical for vast design spaces with up to 10^{30} candidates. Other methods, such as BODi, employ hill-climbing local search to optimize acquisition functions for selecting subsequent candidates, a process known for its prolonged duration. To tackle these challenges, we utilize a PSL model to directly generate predicted Pareto optimal solutions based on

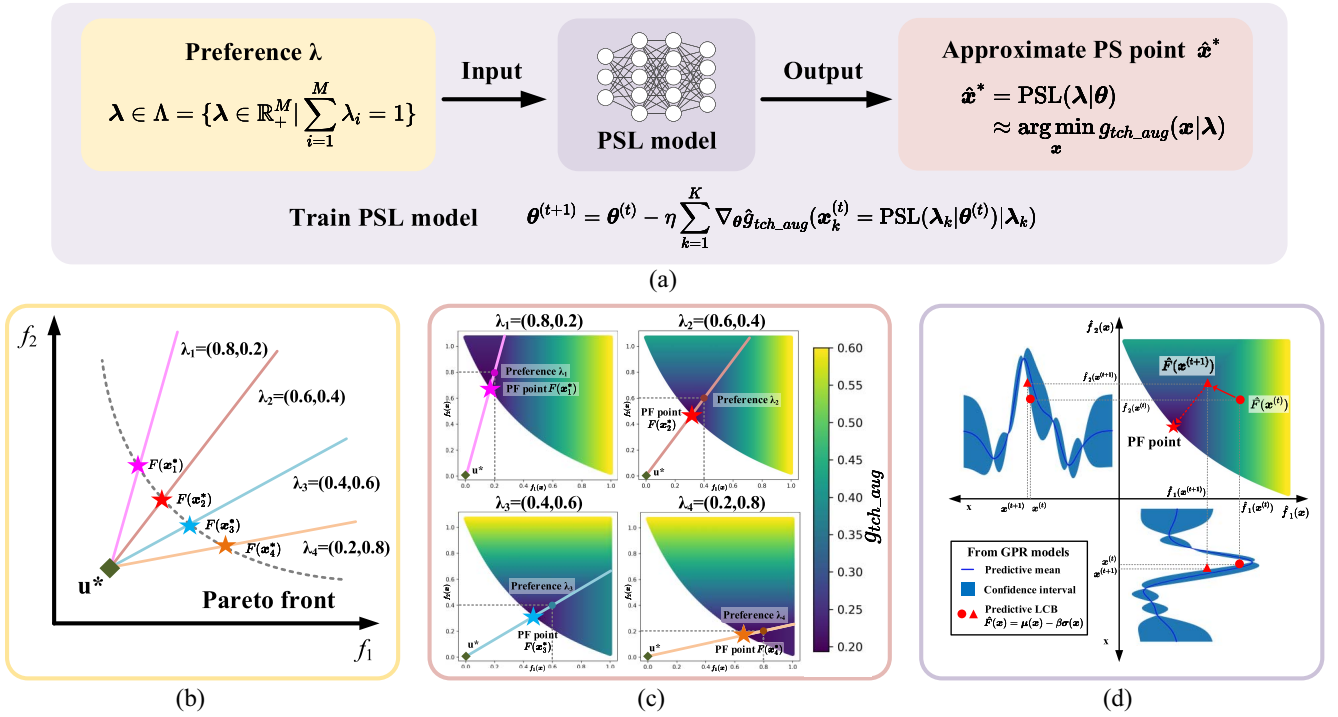


Fig. 3. Illustrations of (a) the PSL model, (b) the preference $\lambda_1, \dots, \lambda_4$, (c) the augmented Tchebycheff function g_{tch_aug} under $\lambda_1, \dots, \lambda_4$, and (d) the PSL model training. The PSL model, implemented as an MLP, can generate an approximate PS point $\hat{x}^*$ tailored to a QoR preference λ . For a specific preference λ , the corresponding augmented Tchebycheff function g_{tch_aug} attains its minimum value at the real PS point x^* , which is approximated by $\hat{x}^*$. The PSL model is iteratively updated by ∇g_{tch_aug} . To reduce the number of actual simulations, APPLE-DSE employs the LCB values of GPR models as surrogates to calculate $\hat{g}_{tch_aug}$. (a) Overview of PSL model. (b) Select preferences. (c) Get PS/PF points by preferences. (d) Train PSL model by GPR models.

predefined preferences, significantly enhancing optimization efficiency.

In real-world design scenarios, designers often possess a set of existing Pareto optimal design points. However, when QoR objectives change, necessitating the acquisition of new designs, existing methods fall short. For instance, consider Fig. 3(b), where $\mathbf{x}_1^*$ and $\mathbf{x}_4^*$ represent existing Pareto-optimal design points. If a designer seeks a design that outperforms $\mathbf{x}_1^*$ in f_2 and $\mathbf{x}_4^*$ in f_1 , conventional DSE methods struggle to address this challenge. This limitation stems from their focus solely on the design parameter space, without considering the corresponding QoR space. In contrast, APPLE-DSE effectively tackles this issue. As depicted in Fig. 3(a), leveraging QoR preferences, the PSL model generates Pareto optimal points tailored to these preferences. Consequently, by setting preference values in f_2 and f_1 to exceed those of $\mathbf{x}_1^*$ and $\mathbf{x}_4^*$, respectively, the resulting Pareto optimal designs from the PSL model, such as $\mathbf{x}_2^*$ and $\mathbf{x}_3^*$, satisfy the updated QoR requirements.

For MOO problems, the preference $\lambda = \{\lambda_i | i = 1, \dots, M\}$ is a weight vector on M objectives, where each λ_i is positive and the sum of λ_i is 1. The augmented Tchebycheff approach [23], [24], [25] can map each PS point $\mathbf{x}^*$ to a unique preference λ for M objective functions. PS point $\mathbf{x}^*$ and preference λ correspond one-to-one

$$\mathbf{x}^*(\lambda | \lambda \in \Lambda) = \arg \min_{\mathbf{x}} g_{tch_aug}(\mathbf{x} | \lambda) \quad (8)$$

$$\Lambda = \{\lambda \in \mathbb{R}_+^M | \sum_{i=1}^M \lambda_i = 1\} \quad (9)$$

$$g_{tch_aug}(\mathbf{x} | \lambda) = \max_{1 \leq i \leq M} \{\lambda_i (f_i(\mathbf{x}) - u_i)\} + \rho \sum_{i=1}^M \lambda_i f_i(\mathbf{x}) \quad (10)$$

where u_i is the ideal minimum value (utopia value) of f_i . In this work, $u_i = z_{i\min} - 0.1|z_{i\min}|$ is dynamically set based on the current best value $z_{i\min}$ for each QoR. ρ distinguishes the Pareto optimal value in cases where the first term on the right-hand side of (10) equals the same value. We set $\rho = 0.001$. As depicted in Fig. 3(c), the g_{tch_aug} values in the QoR space differ based on different preferences. The point with the least g_{tch_aug} is the PS point with a certain preference.

As shown in Fig. 3(a), APPLE-DSE models the relationship between $\mathbf{x}^*$ and λ by the PSL model, which is implemented as a multilayer perceptron (MLP) neural network

$$\hat{\mathbf{x}}^*(\lambda) = \text{PSL}(\lambda | \theta) \approx \mathbf{x}^*(\lambda) \quad (11)$$

$$\theta^{(t+1)} = \theta^{(t)} - \eta \sum_{k=1}^K \nabla_{\theta} \hat{g}_{tch_aug}(\mathbf{x}_k^{(t)} = \text{PSL}(\lambda_k | \theta^{(t)}) | \lambda_k) \quad (12)$$

$$\hat{g}_{tch_aug}(\mathbf{x} | \lambda) = \max_{1 \leq i \leq M} \{\lambda_i (\hat{f}_i(\mathbf{x}) - u_i)\} + \rho \sum_{i=1}^M \lambda_i \hat{f}_i(\mathbf{x}) \quad (13)$$

where θ represents the neural network parameters and the model optimizer is Adam with learning rate $\eta = 0.001$. The neural network has two hidden layers, each of which contains 256 neurons. The activation function is the ReLU function. The size of the input layer is the objective number M and the size of the output layer is the design parameter number N . The PSL model is iteratively updated by gradients of g_{tch_aug} .

Details regarding the PSL model update will be discussed in Section III-C.

To compute g_{tch_aug} , the QoR values are required. Since obtaining the true QoR metrics requires time-consuming simulations, APPLE-DSE employs GPR models as surrogates for the actual simulation, as Fig. 3(d) shows.

To explore a wider range of potential solutions, APPLE-DSE utilizes the lower-confidence bound (LCB) as the predicted QoR value, which is defined in

$$\hat{f}_i(\mathbf{x}) = \hat{\mu}_i(\mathbf{x}) - \beta \hat{\sigma}_i(\mathbf{x}) \quad (14)$$

where $\hat{\mu}_i(\cdot)$ and $\hat{\sigma}_i(\cdot)$ are obtained by the GPR model. The hyper-parameter β controls the tradeoff between exploring unknown areas of the search space and exploiting the local optimum. Because of the enormous design space, modeling of the design space inevitably involves errors. The LCB offers a more optimistic estimate, thereby improving optimization efficiency within a large design space and limited time constraints. To discover an improved design within a constrained time budget, we adopt a greedy $\beta = 0.1$.

While the accuracy of various models may vary across different cases, the limited number of known points hinders the development of precise models within the expansive design space of the DSE with the VLSI flow. Hence, we opt for the GPR models to guide our optimization efforts. We will discuss the differences in accuracy among various models in Section IV-B. Additionally, the GPR model facilitates the computation of derivative information crucial for updating the PSL model.

In summary, we provide the pseudo-code of PSL in Algorithm 1. Considering both training time and model accuracy comprehensively, the number of the steps in PSL training phase N_{step} is set as 1000, the number of the random preferences in PSL training phase N_λ is set as 10, and the number of candidate points N_{cand} is set as 1000.

C. Probabilistic Reparameterization

As shown in (12), the PSL model requires gradients for iterative updates. However, the DSE problem involves discrete variables and the gradients cannot be obtained directly. To address this, APPLE-DSE employs the PR technique, which relaxes the original optimization problem.

PR introduces a discrete probability distribution $p(\mathbf{x}|\mathbf{v})$ over design parameters $\mathbf{x}$, parameterized by a vector of continuous parameters $\mathbf{v}$

$$\mathbf{x} = \lfloor \mathbf{v} \rfloor + B, \quad B \sim \text{Bernoulli}(\mathbf{v} - \lfloor \mathbf{v} \rfloor). \quad (15)$$

According to the theory in [26], the original minimization problem of $\hat{g}_{tch_aug}$ can be rewritten as follows:

$$\min \hat{g}_{tch_aug}(\mathbf{x}) \iff \min \mathbb{E}_{\mathbf{x} \sim p(\mathbf{x}|\mathbf{v})} [\hat{g}_{tch_aug}(\mathbf{x})]. \quad (16)$$

The expected value of $\hat{g}_{tch_aug}(\mathbf{x})$ under $p(\mathbf{x}|\mathbf{v})$ is differentiable with respect to the design parameters $\mathbf{v}$, and its gradient can be estimated by the Monte Carlo (MC) method [26], [27]

Algorithm 1: PSL

Input: The explored dataset D , the number of the steps in PSL training phase N_{step} , the number of the preferences in PSL training phase N_λ , the number of candidate points N_{cand}

- 1 Construct and train M GPR models with $D = \{X, Y\}$, where $Y = \{y_i \mid i = 1, \dots, M\}$;
- 2 Randomly initialize the MLP model, namely PSL model, $\text{PSL}(\lambda|\theta^{(0)})$;
- 3 **for** $t = 1 \rightarrow N_{step}$ **do** // Train the MLP model
 - 4 Randomly generate N_λ preferences $\lambda^{(t)} \in \Lambda$, where Λ is defined in (9);
 - 5 Use the MLP model $\text{PSL}(\lambda^{(t)}|\theta^{(t-1)})$ to generate approximate Pareto set points $\hat{\mathbf{x}}^*(\lambda^{(t)})$ in (11);
 - 6 Use the GPR models to predict the QoR values $\hat{f}_i(\hat{\mathbf{x}}^*)$ in (14) and calculate the $\hat{g}_{tch_aug}(\hat{\mathbf{x}}^*|\lambda^{(t)})$ in (13);
 - 7 Update the parameters $\theta^{(t)}$ of the MLP model according to (12);
- 8 **end**
- 9 Randomly generate N_{cand} preferences $\lambda_{cand} \in \Lambda$;
- 10 Obtain the candidate points $\mathbf{x}_{cand}$ by the MLP model $\text{PSL}(\lambda_{cand}|\theta^{(N_{step})})$;

Output: Candidate points $\mathbf{x}_{cand}$

$$\begin{aligned} & \nabla_{\mathbf{v}} \mathbb{E}_{\mathbf{x} \sim p(\mathbf{x}|\mathbf{v})} [\hat{g}_{tch_aug}(\mathbf{x})] \\ &= \frac{1}{N} \sum_{j=1}^J \hat{g}_{tch_aug}(\tilde{\mathbf{x}}_j) \nabla_{\mathbf{v}} \log p(\tilde{\mathbf{x}}_j|\mathbf{v}) \end{aligned} \quad (17)$$

where $\tilde{\mathbf{x}}_j$ is the j th sampled $\mathbf{x}$ from the distribution $p(\mathbf{x}|\mathbf{v})$.

So, the original PSL model can be modified by the PR technique to fit the discrete variables

$$\mathbf{v}^* = \text{PSL}_{PR}(\lambda|\theta_{PR}) \quad (18)$$

$$\theta_{PR}^{(t+1)} = \theta_{PR}^{(t)} - \eta \sum_{k=1}^K \nabla_{\theta_{PR}} \mathbb{E}_{\mathbf{x} \sim p(\mathbf{x}|\mathbf{v})} [\hat{g}_{tch_aug}(\mathbf{x})] \quad (19)$$

$$\begin{aligned} \nabla_{\theta_{PR}} \mathbb{E}[\hat{g}_{tch_aug}(\mathbf{x})] &= \frac{\partial \mathbb{E}[\hat{g}_{tch_aug}(\mathbf{x})]}{\partial \theta_{PR}} \\ &= \frac{\partial \mathbb{E}[\hat{g}_{tch_aug}(\mathbf{x})]}{\partial \hat{\mathbf{f}}} \frac{\partial \hat{\mathbf{f}}}{\partial \mathbf{v}} \frac{\partial \mathbf{v}}{\partial \theta_{PR}} \end{aligned} \quad (20)$$

where $\partial \mathbb{E}[\hat{g}_{tch_aug}(\mathbf{x})]/\partial \hat{\mathbf{f}}$ can be calculated by (17) and the method described in [28], $\partial \hat{\mathbf{f}}/\partial \mathbf{v}$ can be easily calculated according to (6) and (7), and $\partial \mathbf{v}/\partial \theta_{PR}$ can be obtained through neural network back-propagation of the PSL model.

As a result, the PSL model generates the continuous vector $\mathbf{v}^*$. APPLE-DSE samples the candidate point $\mathbf{x}_{cand}$ by (15), conditioned on $\mathbf{v}^*$

$$\mathbf{x}_{cand} = \lfloor \mathbf{v}^* \rfloor + B^*, \quad B^* \sim \text{Bernoulli}(\mathbf{v}^* - \lfloor \mathbf{v}^* \rfloor). \quad (21)$$

D. Simulation Time-Aware Asynchronous Parallel Scheduling Strategy

Design quality evaluations are inherently time-consuming, exacerbated by the fact that different design parameter configurations yield varying simulation times. For instance, using

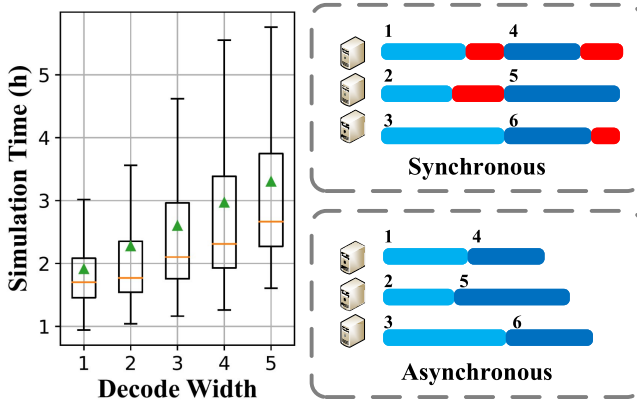


Fig. 4. Simulation time of BOOM versus decode width and an illustration of synchronous and asynchronous parallel. The box plot on the left is generated from the simulations of 5000 randomly generated RISC-V BOOM design points. The green triangles represent the average simulation time, while the orange lines represent the median. The simulation time encompasses RTL generation, RTL simulation, and logic synthesis processes. The blue bars mean the time taken by the simulations and the red bars mean the idle time gaps which are wasted on waiting for the slowest simulation. For RISC-V microarchitecture DSE, different design parameter configurations lead to various simulation times. Asynchronous parallel strategy can improve the efficiency by avoiding the idle time gaps.

the BOOM microarchitecture as an example, the simulation time (encompassing RTL generation, RTL simulation, and logic synthesis processes) increases as the *DecodeWidth* is increased, as shown in Fig. 4. Existing synchronous parallel methods require completion of all simulations before beginning a new iteration. This results in the wastage of idle time gaps, illustrated by the red bars in Fig. 4, thereby diminishing the efficiency of DSE.

To tackle this challenge, APPLE-DSE incorporates a simulation time-aware asynchronous parallel scheduling strategy aimed at bolstering algorithm efficiency. A significant challenge in asynchronous parallelism is avoiding the duplication of candidate point selections. To address this obstacle, APPLE-DSE employs a strategy of selecting distinct preferences to generate a diverse set of candidate points. The preferences may be either artificially specified or randomly generated. The candidates are then ranked using the HVI function, as shown in

$$\text{HVI}(\mathbf{x}^* | \text{PF}, \mathbf{r}) = \text{HV}(\text{PF} \cup \hat{F}(\mathbf{x}^*), \mathbf{r}) - \text{HV}(\text{PF}, \mathbf{r}) \quad (22)$$

where PF and $\mathbf{r}$ represent the PF and the reference point, which are defined consistently with (5). Additionally, for pending simulation points, their predicted values obtained from GPR models are leveraged to construct a pseudo PF. This strategy avoids duplicate simulation points, because the previously selected points have no HVI for the current PF.

In practice, the time allotted for microarchitecture DSE is limited. If the total time budget for DSE is exhausted before all simulations are completed, any ongoing simulations are terminated without consideration. Therefore, maximizing the efficient utilization of the allocated time budget is crucial. APPLE-DSE tackles this challenge by leveraging a GPR model to estimate the simulation time for various points within

the design space, aiding in the identification of points $\mathcal{X}_{\text{cand}}$ that align with the time budget constraint

$$\mathcal{X}_{\text{cand}} = \{\mathbf{x} \in \mathcal{X}_{\text{cand}} | \hat{T}_{\text{sim}}(\mathbf{x}) \leq T_{\text{budget}}\}. \quad (23)$$

The training and prediction processes for simulating time predictions are analogous to those used in PPA predictions with GPR models. We utilize the simulation times of the explored points as our training set, with the LCB serving as an estimate for the simulation time.

As shown in (24), among these feasible points $\mathcal{X}_{\text{cand}}$, the selection process prioritizes the one with the highest HVI for simulation. In scenarios where all points are projected to exceed the time budget, the point with the lowest-predicted simulation time is chosen as an alternative

$$\mathbf{x}_{\text{sim}} = \begin{cases} \arg \max_{\mathbf{x} \in \mathcal{X}_{\text{cand}}} \text{HVI}(\mathbf{x}), & \text{if } \exists \mathbf{x} \in \mathcal{X}_{\text{cand}} \\ \arg \min_{\mathbf{x} \in \mathcal{X}_{\text{cand}}} \hat{T}_{\text{sim}}(\mathbf{x}), & \text{otherwise.} \end{cases} \quad (24)$$

IV. EXPERIMENTAL RESULTS

A. Experimental Setup

In the experiments, we explore the design space for a RISC-V BOOM [7] core and a Rocket core [8]. Both of them are RV64GC configurations. The PPA results are assessed using SoCs in all experiments, each equipped with either the BOOM core or Rocket core as their processor cores. The peripheral modules of these SoCs are standard components provided by the Chipyard framework [4]. A 7nm ASAP7 process design kit (PDK) [29] is employed in the VLSI flow. Given that ASAP7-based designs are not manufacturable, this article only achieves the front-end design in the experiments. The design space includes 22 parameters for the BOOM microarchitecture, nine parameters for the Rocket microarchitecture, and 31 parameters for logic synthesis, as shown in Table I. All the parameters are discrete values. In other words, the design space dimensionalities for BOOM and Rocket are 53 and 40, respectively. The sizes of the design space for BOOM and Rocket are approximately 6.96×10^{32} and 4.06×10^{25} , respectively.

In the VLSI flow, an RTL design is generated from the microarchitecture parameters by Chipyard framework [4], first. Then, an open-source simulator Verilator [30] simulates the design to obtain CPI. Thirteen benchmarks are selected to evaluate the performance of the design: 1) *dhrystone*; 2) *median*; 3) *mm*; 4) *mt-matmul*; 5) *mt-vvadd*; 6) *multiply*; 7) *qsort*; 8) *rsort*; 9) *spmv*; 10) *towers*; 11) *vvadd*; 12) *fir2dim*; and 13) *whetstone*. The first 11 benchmarks are from riscv-test, a benchmark suite by RISC-V International designed to evaluate RISC-V processor performance. These benchmarks can well test RISC-V instructions and are utilized in BOOM-Explorer [14] and GRL-DSE [16]. BOOM-Explorer [14] also utilizes *fir2dim*² and *whetstone*³ to evaluate digital signal processing (DSP) and floating-point performance, respectively. Therefore, we employ them to assess our processor's capabilities. The average CPI of benchmarks is taken as the performance metric. Finally, Cadence Genus Synthesis

¹<https://github.com/riscv-software-src/riscv-tests>

²<https://www.ice.rwth-aachen.de/research/tools-projects/closed-projects/dspstone>

³<https://netlib.org/benchmark/whetstone.c>

TABLE I
DESIGN PARAMETERS

Class	N	Name	Value
BOOM	22	BPD	Boom2,TAGEL,Alpha21264
		fetchWidth	4,8
		numFetchBufferEntries	8,16,24,32,35,40
		numRasEntries	16,24,32
		maxBrCount	8,12,16,20
		decodeWidth	1,2,3,4,5
		numRobEntries	32,64,96,128,130
		numIntPhysRegisters	48,64,80,96,112
		memIssueWidth	1,2
		intIssueWidth	1,2,3,4,5
		numLdqEntries	8,16,24,32
		enablePrefetching	false,true
		enableSFBOP	false,true
		numRXQEntries	2,4,8
		numRCQEntries	8,16,32
		nL2TLBEntries	128,256,512,1024
		nL2TLBWays	1,2,4,8
		nICacheWays	2,4,8
		nICacheTLBWays	8,16,32
		nDCacheWays	2,4,8
		nDCacheMSHRs	2,4,8
		nDCacheTLBWays	8,16,32
Rocket	9	mulUnroll	1,2,4,8
		divUnroll	1,2,4,8
		mulEarlyOut	false,true
		divEarlyOut	false,true
		nDCacheWays	1,2,4
		nDCacheTLBWays	4,8,16
		nDCacheMSHRs	1,2,4
		nICacheWays	1,2,4
		nICacheTLBWays	4,8,16
Genus	31	time_recovery_arcs	false,true
		auto_partition	true,false
		dp_analytical_opt	off,standard,extreme
		dp_area_mode	false,true
		dp_csa	basic,none
		dp_rewriting	basic,advanced,none
		dp_sharing	advanced,basic,none
		exact_match_seq_async_ctrls	false,true
		exact_match_seq_sync_ctrls	false,true
		iopt_enable_floating_output_check	false,true
		iopt_force_constant_removal	false,true
		iopt_lp_power_analysis_effort	low,medium,high
		lbr_seq_in_out_phase_opto	false,true
		optimize_net_area	true,false
		retime_effort_level	low,medium,high
		retime_optimize_reset	false,true
		syn_generic_effort	low,medium,high
		syn_map_effort	low,medium,high
		syn_opt_effort	low,medium,high
		leakage_power_effort	low,medium,high
		lp_power_analysis_effort	low,medium,high
		enable_latch_max_borrow	false,true
		latch_max_borrow	1:500:1
		enable_max_fanout	false,true
		max_fanout	2:64:1
		enable_lp_clock_gating_max_flops	false,true
		lp_clock_gating_max_flops	8,16,32
		enable_lp_power_optimization_weight	false,true
		lp_power_optimization_weight	0.1:0.9:0.1
		max_dynamic_power	3.0:150.0:0.1
		max_leakage_power	5.0:100.0:0.1

Solution 19.10 is used as the logic synthesis tool to report timing information and estimate power and area metrics. The timing information is the worst-negative slack (WNS). The power metric is the total power. The clock frequency is set as 250 MHz. Design points that fail to meet the timing constraints are categorized as failure points. In our experiments, the PSs obtained by each method do not include any failure points. The PPAs of the failure points are dominated by those of the PSs. All experiments are conducted on a Linux workstation equipped with 80 Intel Xeon Gold 6230 CPUs.

We compare APPLE-DSE with four state-of-the-art methods: 1) BOOM-Explorer [14], [31], a BO-based method for

microarchitecture DSE with a microarchitecture-aware initialization approach MicroAL; 2) GRL-DSE [16], another BO method for microarchitecture DSE which uses graph represent learning GNN to map the design space into graph embedding space; 3) BODi [19], a high-dimensional BO method based on dictionary embedding for discrete optimization problem; and 4) REMOTune [20], a recent local BO method for high-dimensional microarchitecture DSE that combines the trust regions and random embedding. We extend the original BODi from SOO to MOO by using the EHVI acquisition function [32]. Because BOOM-Explorer and REMOTune support parallel simulation in a batch, we compare BOOM-Explorer and REMOTune in both sequential and batch modes.

To ensure a fair comparison of sequential algorithms and demonstrate the effectiveness of asynchronous parallelism, we implement three versions of APPLE-DSE: 1) APPLE-DSE (Seq), a sequential APPLE-DSE method; 2) APPLE-DSE (Sync), a synchronous parallel APPLE-DSE method without simulation time-aware asynchronous parallel scheduling strategy; and 3) APPLE-DSE (Async), the full version of APPLE-DSE. In this article, we label the batch size of the parallel methods after their name. The batch size in the experiments is set to 4. All algorithms are implemented in Python.

IC design houses often maintain a repository of previously developed designs, seeking to explore enhanced PF within stringent time constraints. To address such optimization scenarios, all algorithms are initialized with a standardized set of 100 designs. These designs are sampled from the design space using the MicroAL algorithm as featured in BOOM-Explorer. The experiments are executed within a maximum time budget of one day (86 400 s). The HV indicator serves as a benchmark metric for evaluating the quality of the solutions generated by the algorithms. For HV computation, the reference point is defined as the maximum observed values of CPI, power, and area within the initial dataset. In addition to HV, we also use average distance to reference set (ADRS) to provide a comprehensive evaluation of optimization results

$$ADRS(\Gamma, \Omega) = \frac{1}{|\Gamma|} \sum_{\gamma \in \Gamma} \min_{\omega \in \Omega} d(\gamma, \omega) \quad (25)$$

where $d(\cdot, \cdot)$ is the Euclidean distance function. Γ is the real PF and Ω is the optimized PF. However, due to the vastness of our design space, it is not feasible to obtain a real PF. To address this issue, we consider all the points evaluated in the experiments as the total dataset, utilizing the PF derived from it as the reference set to calculate the $\bar{ADRS}$. The experiments are conducted five times, and the average results are subsequently reported.

Due to the challenges posed by the untraversable high-dimensional design space, our implementations of BOOM-Explorer and GRL-DSE randomly select sample points from the entire design space, effectively creating a limited design space. Since these methods specifically target the RISC-V BOOM microarchitecture design space, which contains 15 633 design points [16], we set the design space size at 15 000. Furthermore, both BOOM-Explorer and GRL-DSE necessitate the evaluation of each point in the design space

TABLE II
COMPARISON OF MODELING ACCURACY AMONG DIFFERENT MODELS

Model	BOOM				Rocket			
	PPAs		sim_time		PPAs		sim_time	
	MSE (Train)	MSE (Test)	MSE (Train)	MSE (Test)	MSE (Train)	MSE (Test)	MSE (Train)	MSE (Test)
MLP	9.7662E-03	3.8623E-01	1.0343E-03	1.4593E-01	3.9221E-03	1.2709E-01	2.7471E-03	4.6643E-01
KNN	4.0286E-01	1.0075E+00	5.2663E-01	5.4184E-02	5.5776E-01	3.8697E-01	6.3534E-01	5.8998E-01
Ridge	6.8157E-02	3.7048E-01	1.9271E-01	2.8181E-01	8.9495E-02	1.0625E-01	2.1926E-01	4.2511E-01
SVM	1.2180E-01	6.8322E-01	4.1576E-01	4.1993E-02	1.2453E-01	1.9354E-01	1.4248E-01	6.3307E-01
RF	3.0413E-02	2.2069E-01	8.0321E-02	3.2268E-02	2.2582E-02	3.9382E-02	6.5047E-02	5.0067E-01
AdaBoost	5.6348E-02	2.8306E-01	6.2375E-02	8.0836E-02	5.1373E-02	7.3022E-02	1.1810E-01	4.9039E-01
XGBoost	8.3155E-07	1.7532E-01	2.6938E-07	3.4530E-02	2.3621E-07	2.6984E-02	2.8717E-07	5.9937E-01
GPR	1.0983E-14	1.3525E-01	1.3776E-19	1.9308E-02	5.2806E-19	1.3530E-02	4.1761E-20	2.4510E-01

TABLE III
COMPARISON BETWEEN GPR MODELS AND XGBOOST MODELS FOR MICROARCHITECTURE DSE

Algorithm	APPLE-DSE-4 (Async)					
Circuit	BOOM			Rocket		
Surrogate model	XGBoost	GPR	Improvement	XGBoost	GPR	Improvement
↑ HV	174,004	194,016	11.50%	6,570	7,535	14.69%
↓ ADRS	1,192	135	88.67%	37.44	16.73	55.32%
↓ Min. CPI	0.8044	0.7853	2.37%	1.6143	1.6070	0.45%
↓ Min. Power	13.621mW	8.285mW	39.17%	3.753mW	3.561mW	5.12%
↓ Min. Area	75,710um ²	66,540um²	12.11%	57,759um ²	57,541um²	0.04%
↓ Time/iter	31.3s	33.6s	-7.34%	31.5s	30.7s	2.54%
# Simulation	37	65	75.68%	90	104	13.46%
↑ Min. WNS	0ps	0ps	1.00×	1ps	22ps	22.00×
↑ Max. WNS	20ps	267ps	13.35×	817ps	868ps	1.06×

to select the subsequent point for simulation. Consequently, a larger design space results in longer times required to obtain the next candidate point, thereby limiting exploration within a constrained time budget. As a result, we opt not to use a larger design space.

B. Model Accuracy Comparison

Due to the time-consuming simulations, we employ surrogate models to substitute for actual simulations when training the PSL model. Common surrogate models include MLP, k -nearest neighbors (KNN), Ridge Regression, support vector machines (SVM), RF, AdaBoost, XGBoost, and GPR. With the exception of XGBoost [33], which is implemented using its open-source code, all other models are executed with scikit-learn [34]. To enhance the accuracy of the modeling, we normalize the input variables and standardize the outputs.

Table II shows the results of model accuracy comparison. We use the same initial dataset as the training set, which consists of 100 points, while the test set comprises points that can be obtained within a one-day time budget. On the training set, all models achieve relatively small mean squared errors (MSEs), with the GPR model significantly outperforming the others, particularly due to its advantages in modeling small sample sizes. On the test set, the accuracy of the model is limited due to the considerably smaller size of the training set in relation to the overall design space; however, the GPR model continues to outperform the other models. Furthermore,

the GPR model provides uncertainty estimates, with all true values falling within the predicted range of $\mu \pm \sigma$.

We use the second-best model, XGBoost, as the surrogate model for comparative experiments. The results are shown in Table III. *Min. CPI*, *Min. Power*, and *Min. Area* in the table denote the minimum values of CPI, power, and area achieved by the optimization. *Min. WNS* and *Max. WNS* refer to the minimum and maximum values of WNS within PSs. *Time/iter* denotes the algorithm run time per iteration, excluding simulation time. In this article, we refer to the time elapsed from inputting design parameters to obtaining PPA results as simulation time, which includes RTL generation, RTL simulation, and logic synthesis processes. It is evident that the GPR models outperform the XGBoost models in both MOO metrics and PPAs. Consequently, we select the GPR models as our surrogate models.

C. RISC-V BOOM Microarchitecture DSE

The optimization results are shown in Table IV and Fig. 5. Compared with the state-of-the-art methods, APPLE-DSE obtains the largest HV and the least ADRS in both sequential and batch modes. The sequential APPLE-DSE-1 (Seq) achieves 4.35% ~ 4.64% improvement in HV and 49.07% ~ 73.30% improvement in ADRS compared with BOOM-Explorer-1, GRL-DSE, BODi, and REMOTune-1. This indicates that PSL has higher efficiency in exploring the PF than the baselines. APPLE-DSE-4 (Async) achieves

TABLE IV
OPTIMIZATION RESULTS AND CORRESPONDING TIME FOR RISC-V BOOM MICROARCHITECTURE DSE

Algorithm	BOOM-Explorer-1 [14]	BOOM-Explorer-4 [31]	GRL-DSE [16]	BODi [19]	REMOTune-1 [20]	REMOTune-4 [20]	APPLE-DSE-1 (Seq)	APPLE-DSE-4 (Sync)	APPLE-DSE-4 (Async)
↑ HV	172,391	172,391	172,540	172,871	172,428	172,551	180,390	186,757	194,016
Improvement	0.00%	0.00%	0.09%	0.28%	0.02%	0.09%	4.64%	8.33%	12.54%
↓ ADRS	669	455	1,276	1,034	1,114	1,289	340	283	135
Improvement	0.00%	31.99%	-90.79%	-54.55%	-66.59%	-92.65%	49.07%	57.68%	79.73%
↓ Min. CPI	1.3919	1.4186	0.9080	0.9312	1.0531	0.9501	0.8813	0.8357	0.7853
Improvement	0.00%	-1.91%	34.77%	33.10%	24.34%	31.74%	36.69%	39.96%	43.58%
↓ Min. Power	10.763mW	10.456mW	13.440mW	10.048mW	11.945mW	12.274mW	8.326mW	8.418mW	8.285mW
Improvement	0.00%	2.85%	-24.88%	6.64%	-10.98%	14.04%	22.64%	21.78%	23.02%
↓ Min. Area	68,831um ²	67,109um ²	77,198um ²	71,035um ²	73,364um ²	74,317um ²	66,800um ²	66,779um ²	66,540um²
Improvement	0.00%	2.50%	-12.16%	-3.20%	-6.59%	-7.97%	2.95%	2.98%	3.33%
↓ Time/iter	3,005.9s	2,005.1s	536.3s	950.5s	94.4s	104.7s	23.5s	27.4s	33.6s
Speed up	1.00×	1.50×	5.60×	3.16×	31.84×	28.70×	127.73×	109.67×	89.43×
# Simulation	11	40	8	11	13	42	16	56	65
Multiple	1.00×	3.64×	0.73×	1.00×	1.18×	3.82×	1.45×	5.09×	5.91×
Min. WNS	4ps	17ps	0ps	0ps	0ps	0ps	0ps	0ps	0ps
Max. WNS	142ps	145ps	68ps	41ps	10ps	3ps	8ps	264ps	267ps

TABLE V
PARETO OPTIMAL DESIGNS FOR RISC-V BOOM MICROARCHITECTURE

Design	Small BOOM [7]	BODi [19]	REMOTune-4 [20]	APPLE-DSE-4 (Async)	Medium BOOM [7]	BODi [19]	REMOTune-4 [20]	APPLE-DSE-4 (Async)
↓ CPI	1.3725	1.2279	1.1680	1.1676	0.9906	1.0579	1.0618	0.9741
Improvement	0.00%	10.54%	14.90%	14.93%	0.00%	-6.79%	-7.18%	1.67%
↓ Power	13.844mW	10.382mW	10.207mW	10.080mW	17.018mW	12.907mW	13.413mW	12.830mW
Improvement	0.00%	25.01%	26.27%	27.19%	0.00%	24.16%	21.19%	24.61%
↓ Area	72,200um ²	70,546um ²	70,973um ²	70,097um²	77,582um ²	74,431um ²	74,562um ²	74,307um²
Improvement	0.00%	2.29%	1.70%	2.91%	0.00%	4.06%	3.89%	4.22%
WNS	98ps	33ps	3ps	0ps	68ps	0ps	0ps	2ps

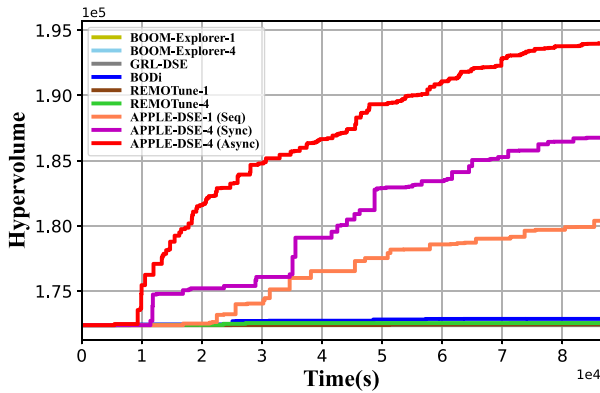


Fig. 5. Optimization results for RISC-V BOOM microarchitecture DSE.

12.23% ~ 12.54% improvement in HV and 71.11% ~ 89.48% improvement in ADRS compared with BOOM-Explorer-1, BOOM-Explorer-4, GRL-DSE, BODi, REMOTune-1, and REMOTune-4. In addition, APPLE-DSE-4 (Async) achieves 7.55% and 3.89% improvement in HV and 60.21% and 52.11% improvement in ADRS compared with APPLE-DSE-1 (Seq) and APPLE-DSE-4 (Sync). This demonstrates the effectiveness of asynchronous parallelism in the microarchitecture DSE problem.

In terms of PPA metrics, APPLE-DSE-4 (Async) gains improvements of 6.03% ~ 44.64% in CPI, 0.50% ~ 38.36% in power, and 0.36% ~ 13.81% in area, compared to BOOM-Explorer-1, BOOM-Explorer-4, GRL-DSE, BODi, REMOTune-1, REMOTune-4, APPLE-DSE-1 (Seq) and APPLE-DSE-4 (Sync).

Besides, APPLE-DSE-1 (Seq) takes the least algorithm time per iteration among all methods. Since the time complexity of the GPR models is $O(N^3)$, where N is the number of data points, APPLE-DSE-4 (Async) takes more time because of its more simulation points. Compared to BOOM-Explorer-1, GRL-DSE, BODi, and REMOTune-1, APPLE-DSE-1 (Seq) achieves a speed-up of $4.01\times \sim 127.73\times$ in algorithm time per iteration. Compared to BOOM-Explorer-4 and REMOTune-4, APPLE-DSE-4 (Async) achieves a speed-up of $59.67\times$ and $3.12\times$ in algorithm time per iteration.

To verify the effectiveness of optimization, we compare the Pareto optimal designs obtained from the experiment with the official BOOM designs (*Small BOOM* and *Medium BOOM*) from [7]. These Pareto optimal designs are the individual designs selected from the PF optimized through our methods and baselines. These designs are obtained during the optimization process. As Table V shows, APPLE-DSE-4 (Async) achieves improvements of 14.93% in CPI, 27.19% in

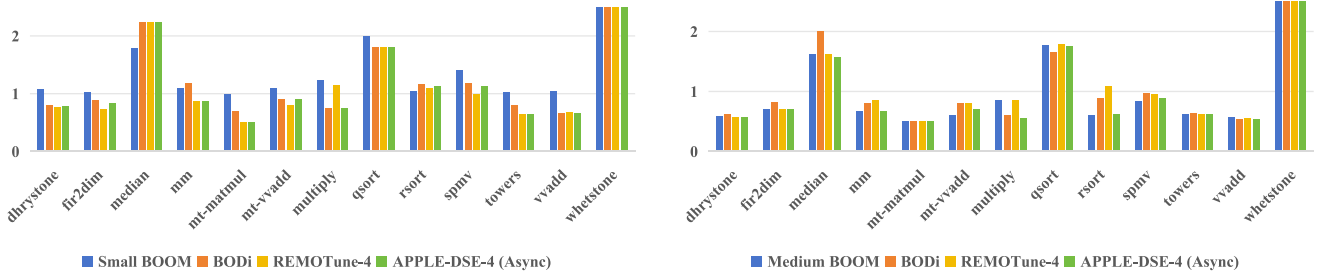


Fig. 6. CPIs on benchmarks for the Pareto optimal designs of the RISC-V BOOM microarchitecture obtained by BODi [19], REMOTune-4 [20], and APPLE-DSE-4 (Async).

TABLE VI
OPTIMIZATION RESULTS AND CORRESPONDING TIME FOR ROCKET MICROARCHITECTURE DSE

Algorithm	BOOM- Explorer-1 [14]	BOOM- Explorer-4 [31]	GRL-DSE [16]	BODi [19]	REMOTune-1 [20]	REMOTune-4 [20]	APPLE-DSE-1 (Seq)	APPLE-DSE-4 (Sync)	APPLE-DSE-4 (Async)
↑ HV	6,451	6,451	6,475	6,497	6,515	6,539	7,190	7,360	7,535
Improvement	0.00%	0.00%	0.38%	0.72%	1.00%	1.37%	11.47%	14.09%	16.81%
↓ ADRS	55.72	55.74	52.09	52.64	50.78	47.45	31.78	21.12	16.73
Improvement	0.00%	-0.04%	6.51%	5.52%	8.87%	14.84%	42.96%	62.09%	69.96%
↓ Min. CPI	1.9192	1.9267	1.6104	1.6121	1.6828	1.6768	1.6072	1.6103	1.6070
Improvement	0.00%	-0.39%	16.09%	16.00%	12.32%	12.63%	16.26%	16.09%	16.27%
↓ Min. Power	4.756mW	4.794mW	3.935mW	3.919mW	3.877mW	4.108mW	3.629mW	3.634mW	3.561mW
Improvement	0.00%	-0.80%	17.26%	17.60%	18.48%	13.63%	23.69%	23.59%	25.12%
↓ Min. Area	57,886 μm^2	57,886 μm^2	58,031 μm^2	57,950 μm^2	58,039 μm^2	57,847 μm^2	57,626 μm^2	57,580 μm^2	57,541μm^2
Improvement	0.00%	0.00%	-0.25%	-0.11%	-0.26%	0.07%	0.45%	0.53%	0.60%
↓ Time/iter	3,128.9s	2,988.4s	658.9s	998.9s	121.0s	133.2s	25.7s	29.3s	30.7s
Speed up	1.00×	1.05×	4.75×	3.13×	25.87×	23.49×	121.77×	106.46×	101.98×
# Simulation	16	68	22	21	29	90	26	96	104
Multiple	1.00×	4.25×	1.38×	1.31×	1.81×	5.63×	1.63×	6.00×	6.50×
Min. WNS	2ps	2ps	16ps	11ps	4ps	1ps	90ps	24ps	22ps
Max. WNS	228ps	228ps	792ps	770ps	776ps	776ps	787ps	863ps	868ps

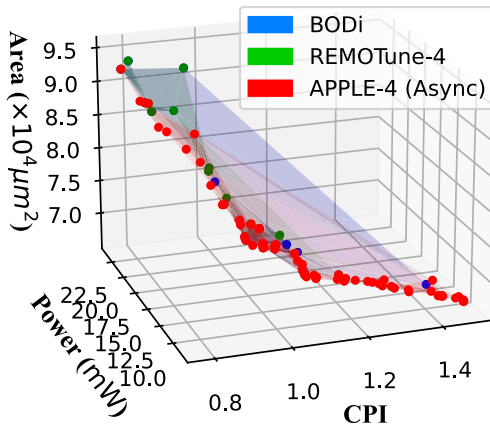


Fig. 7. PFs of RISC-V BOOM obtained by BODi [19], REMOTune-4 [20], and APPLE-DSE-4 (Async).

power, and 2.91% in area compared to Small BOOM, while APPLE-DSE-4 (Async) achieves improvements of 1.67% in CPI, 24.61% in power, and 4.22% in area compared to Medium BOOM. Fig. 6 shows the CPIs on benchmarks for the Pareto optimal designs of the BOOM microarchitecture obtained by BODi, REMOTune-4, and APPLE-DSE-4 (Async). Fig. 7 shows the PFs obtained from a sole run of

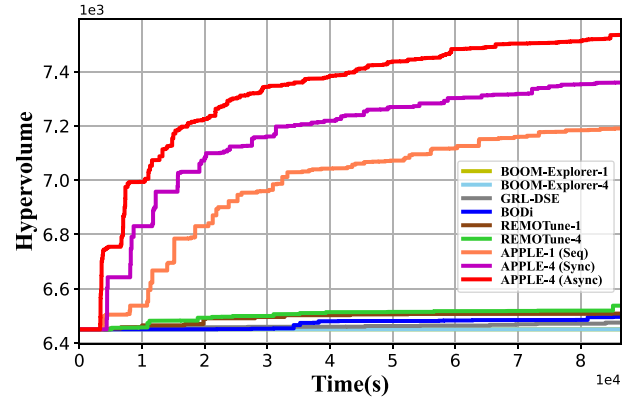


Fig. 8. Optimization results for Rocket microarchitecture DSE.

BODi, REMOTune-4, and APPLE-DSE-4 (Async). APPLE-DSE-4 (Async) gains the best PF compared to BODi and REMOTune-4.

D. Rocket Microarchitecture DSE

The optimization results are shown in Table VI and Fig. 8. Compared with the state-of-the-art methods, APPLE-DSE-4 (Async) obtains the largest HV and the least ADRS in

TABLE VII
PARETO OPTIMAL DESIGNS FOR ROCKET MICROARCHITECTURE

Design	BigCore [8]	BODi [19]	REMOTune-4 [20]	APPLE-DSE-4 (Async)	MedCore [8]	BODi [19]	REMOTune-4 [20]	APPLE-DSE-4 (Async)
↓ CPI	1.6121	1.6121	1.6147	1.6103	2.2670	1.9615	1.9861	1.9201
Improvement	0.00%	0.00%	-0.16%	0.11%	0.00%	13.48%	12.39%	15.30%
↓ Power	5.309mW	4.525mW	4.305mW	3.998mW	5.064mW	3.823mW	4.756mW	3.599mW
Improvement	0.00%	14.76%	18.91%	24.70%	0.00%	24.51%	6.09%	28.93%
↓ Area	60,812um ²	59,677um ²	60,016um ²	59,655um²	57,937um ²	57,861um ²	57,800um ²	57,760um²
Improvement	0.00%	1.87%	1.31%	1.90%	0.00%	0.13%	0.24%	0.31%
WNS	107ps	156ps	458ps	468ps	107ps	98ps	197ps	824ps

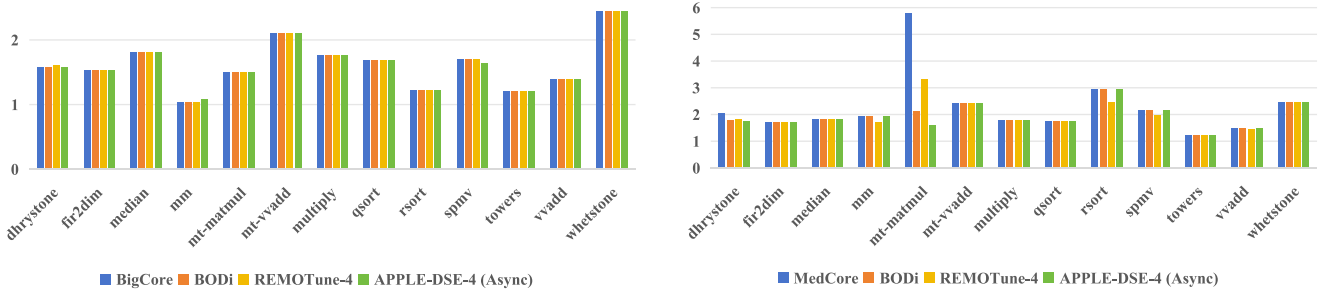


Fig. 9. CPIs on benchmarks for the Pareto optimal designs of the rocket microarchitecture obtained by BODi [19], REMOTune-4 [20], and APPLE-DSE-4 (Async).

both sequential and batch mode. The sequential APPLE-DSE-1 (Seq) improves HV by 10.37% ~ 11.47% and improves ADRS by 37.41% ~ 42.96% over BOOM-Explorer-1, GRL-DSE, BODi, and REMOTune-1. APPLE-DSE-4 (Async) achieves HV improvements of 15.23% ~ 16.81% and ADRS improvements of 64.73% ~ 69.99% compared to BOOM-Explorer-1, BOOM-Explorer-4, GRL-DSE, BODi, REMOTune-1 and REMOTune-4, respectively. Compared to APPLE-DSE-1 (Seq) and APPLE-DSE-4 (Sync), APPLE-DSE-4 (Async) achieves HV improvements of 4.79% and 2.38% and ADRS improvements of 47.34% and 20.77%.

In terms of PPA metrics, APPLE-DSE-4 (Async) achieves improvements of 0.01% ~ 16.59% in CPI, 1.87% ~ 25.72% in power, and 0.07% ~ 0.86% in area, compared to BOOM-Explorer-1, BOOM-Explorer-4, GRL-DSE, BODi, REMOTune-1, REMOTune-4, APPLE-DSE-1 (Seq) and APPLE-DSE-4 (Sync).

Besides, APPLE-DSE-1 (Seq) takes the least algorithm run time per iteration among all methods. Compared to BOOM-Explorer-1, GRL-DSE, BODi, and REMOTune-1, APPLE-DSE-1 (Seq) achieves a speed-up of $4.71 \times \sim 121.77 \times$ in algorithm time per iteration. Compared to BOOM-Explorer-4 and REMOTune-4, APPLE-DSE-4 (Async) achieves a speed-up of $97.34 \times$ and $4.34 \times$ in algorithm time per iteration.

We also compare the Pareto-optimal designs obtained from the experiment with the official Rocket designs (*BigCore* and *MedCore*) from [8]. As Table VII shows, APPLE-DSE-4 (Async) improves CPI by 0.11%, power by 24.70%, and area by 1.90% compared with BigCore, while APPLE-DSE-4 (Async) improves CPI by 15.30%, power by 28.93%, and area by 0.21% compared with MedCore. Fig. 9 shows the CPIs on benchmarks for the Pareto optimal designs of the

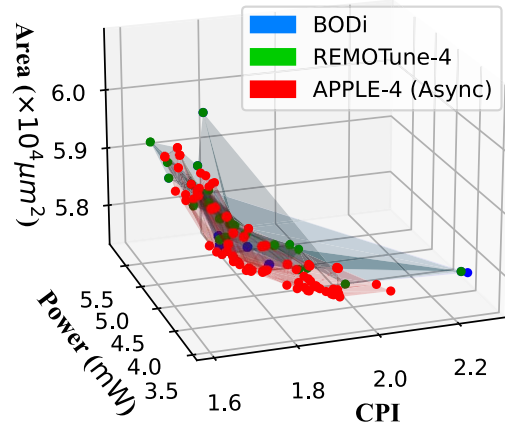


Fig. 10. PFs of Rocket obtained by BODi [19], REMOTune-4 [20], and APPLE-DSE-4 (Async).

Rocket microarchitecture obtained by BODi, REMOTune-4, and APPLE-DSE-4 (Async). Fig. 10 shows the PFs obtained from a sole run of BODi, REMOTune-4, and APPLE-DSE-4 (Async). APPLE-DSE-4 (Async) gains the best PF compared to BODi and REMOTune-4.

E. Discussion

Because of the time-consuming DKL-GP in high dimensions, BOOM-Explorer's run time is much slower than other methods. Furthermore, BOOM-Explorer is not specifically designed for high-dimensional microarchitecture DSE problems. It over-explores the vast design space, unable to improve the HV in a short limited period of time. So, its optimization results are subpar, and the batch mode fails

to achieve a higher HV compared to the sequential mode. GRL-DSE gets mediocre HV because acquiring an accurate GNN for the high-dimensional design space is difficult. Since dictionary embedding is better suited for high-dimensional problems, BODi outperforms BOOM-Explorer and GRL-DSE. However, BODi's hill-climbing local search requires more acquisition evaluations than other methods, resulting in a longer optimization time. Although REMOTune is designed for high-dimensional DSE, it achieves worse results than APPLE-DSE because it ignores the relationship between PS points. Additionally, the trust region-based method limits its robustness. APPLE-DSE achieves a better PF by capturing the structural characteristics of the PS. Furthermore, the asynchronous parallel scheduling strategy in APPLE-DSE allows it to explore more points within the limited time budget. Because APPLE-DSE generates candidate points directly from the PSL model without optimizing acquisition functions, the run time of our methods is much less than other methods.

V. CONCLUSION

In this article, we introduce APPLE-DSE, an asynchronous parallel microarchitecture DSE method, which leverages a PSL model to approximate the PF with a lightweight evaluation approach. By capturing the structural characteristics of the PS guided by surrogate models, APPLE-DSE can effectively explore tradeoff areas within the approximate PS. Furthermore, to handle discrete variables, APPLE-DSE incorporates the PR technique, adapting the PSL model accordingly. To enhance optimization efficiency, APPLE-DSE integrates a simulation time-aware asynchronous parallel scheduling strategy. Experimental results show that APPLE-DSE achieves a maximum improvement of 16.81% in HV within the same time budget and a $127.73\times$ speedup in algorithm run time per iteration compared to state-of-the-art methods.

REFERENCES

- [1] M. J. Bass and C. M. Christensen, "The future of the microprocessor business," *IEEE Spectr.*, vol. 39, no. 4, pp. 34–39, Apr. 2002.
- [2] W. Wolf, A. A. Jerraya, and G. Martin, "Multiprocessor system-on-chip (MPSoC) technology," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 27, no. 10, pp. 1701–1713, Oct. 2008.
- [3] K. Asanović and D. A. Patterson, "Instruction sets should be free: The case for RISC-V," Dept. Electr. Eng. Comput. Sci., Univ. California, Berkeley, CA, USA, Rep. UCB/EECS-2014-146, 2014.
- [4] A. Amid et al., "Chipyard: Integrated design, simulation, and implementation framework for custom SoCs," *IEEE Micro*, vol. 40, no. 4, pp. 10–21, Jul./Aug. 2020.
- [5] J. Bachrach et al., "Chisel: Constructing hardware in a scala embedded language," in *Proc. 49th Annu. Design Autom. Conf. (DAC)*, 2012, pp. 1212–1221.
- [6] C. Celio, D. A. Patterson, and K. Asanovic, "The Berkeley out-of-order machine (BOOM): An industry-competitive, synthesizable, parameterized RISC-V processor," Dept. Electr. Eng. Comput. Sci., Univ. California, Berkeley, CA, USA, Rep. UCB/EECS-2015-167, 2015.
- [7] J. Zhao, B. Korpan, A. Gonzalez, and K. Asanovic, "SonicBOOM: The 3rd generation Berkeley out-of-order machine," in *Proc. 4th Workshop Comput. Archit. Res. (RISC-V)*, 2020, pp. 1–7.
- [8] K. Asanovic et al., "The rocket chip generator," Dept. Electr. Eng. Comput. Sci., Univ. California, Berkeley, CA, USA, Rep. UCB/EECS-2016-17, 2016.
- [9] N. Binkert et al., "The gem5 simulator," *ACM SIGARCH Comput. Archit. News*, vol. 39, no. 2, pp. 1–7, 2011.
- [10] H. Liew et al., "Hammer: A modular and reusable physical design flow tool," in *Proc. 59th ACM/IEEE Design Autom. Conf. (DAC)*, 2022, pp. 1335–1338.
- [11] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *IEEE Trans. Evol. Comput.*, vol. 6, no. 2, pp. 182–197, Apr. 2002.
- [12] Q. Zhang and H. Li, "MOEA/D: A multiobjective evolutionary algorithm based on decomposition," *IEEE Trans. Evol. Comput.*, vol. 11, no. 6, pp. 712–731, Dec. 2007.
- [13] H.-Y. Liu and L. P. Carloni, "On learning-based methods for design-space exploration with high-level synthesis," in *Proc. ACM/EDAC/IEEE 50th Annu. Design Autom. Conf. (DAC)*, 2013, pp. 1–7.
- [14] C. Bai, Q. Sun, J. Zhai, Y. Ma, B. Yu, and M. D. F. Wong, "BOOM-explorer: RISC-V BOOM microarchitecture design space exploration framework," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, 2021, pp. 1–9.
- [15] Q. Gautier, A. Althoff, C. L. Crutchfield, and R. Kastner, "Sherlock: A multi-objective design space exploration framework," *ACM Trans. Design Autom. Electron. Syst.*, vol. 27, no. 4, pp. 1–20, 2022.
- [16] X. Yi, J. Lu, X. Xiong, D. Xu, L. Shang, and F. Yang, "Graph representation learning for microarchitecture design space exploration," in *Proc. 60th ACM/IEEE Design Autom. Conf. (DAC)*, 2023, pp. 1–6.
- [17] S. Daulton, D. Eriksson, M. Balandat, and E. Bakshy, "Multi-objective bayesian optimization over high-dimensional search spaces," in *Proc. 38th Conf. Uncertain. Artif. Intell. (UAI)*, 2022, pp. 507–517.
- [18] Z. Wang, F. Hutter, M. Zoghi, D. Matheson, and N. De Freitas, "Bayesian optimization in a billion dimensions via random embeddings," *J. Artif. Intell. Res.*, vol. 55, pp. 361–387, Feb. 2016.
- [19] A. Deshwal, S. Ament, M. Balandat, E. Bakshy, J. R. Dossa, and D. Eriksson, "Bayesian optimization over high-dimensional combinatorial spaces via dictionary-based embeddings," in *Proc. 26th Int. Conf. Artif. Intell. Statist. (AISTATS)*, 2023, pp. 7021–7039.
- [20] S. Zheng, H. Geng, C. Bai, B. Yu, and M. D. F. Wong, "Boosting VLSI design flow parameter tuning with random embedding and multi-objective trust-region Bayesian optimization," *ACM Trans. Design Autom. Electron. Syst.*, vol. 28, no. 5, pp. 1–23, 2023.
- [21] C. Williams and C. Rasmussen, "Gaussian processes for regression," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 1995, pp. 515–520.
- [22] C. E. Rasmussen and C. K. I. Williams, *Gaussian Processes for Machine Learning*. Cambridge, MA, USA: MIT Press, 2005.
- [23] R. E. Steuer and E.-U. Choo, "An interactive weighted Tchebycheff procedure for multiple objective programming," *Math. Program.*, vol. 26, pp. 326–344, Oct. 1983.
- [24] I. Kaliszewski, "A modified weighted Tchebycheff metric for multiple objective programming," *Comput. Oper. Res.*, vol. 14, no. 4, pp. 315–323, 1987.
- [25] J. Knowles, "ParEGO: A hybrid algorithm with online landscape approximation for expensive multiobjective optimization problems," *IEEE Trans. Evol. Comput.*, vol. 10, no. 1, pp. 50–66, Feb. 2006.
- [26] S. Daulton, X. Wan, D. Eriksson, M. Balandat, M. A. Osborne, and E. Bakshy, "Bayesian optimization over discrete and mixed spaces via probabilistic reparameterization," in *Proc. 36th Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2022, pp. 12760–12774.
- [27] J. P. C. Kleijnen and R. Y. Rubinstein, "Optimization and sensitivity analysis of computer simulation models by the score function method," *Eur. J. Oper. Res.*, vol. 88, no. 3, pp. 413–427, 1996.
- [28] X. Lin, Z. Yang, X. Zhang, and Q. Zhang, "Pareto set learning for expensive multi-objective optimization," in *Proc. 36th Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2022, pp. 19231–19247.
- [29] L. T. Clark et al., "ASAP7: A 7-nm finFET predictive process design kit," *Microelectron. J.*, vol. 53, pp. 105–115, Jul. 2016.
- [30] W. Snyder, "Verilator 4.0: Open simulation goes multithreaded," in *Proc. Open Source Digit. Design Conf. (ORConf)*, 2018, pp. 1–31.
- [31] C. Bai, Q. Sun, J. Zhai, Y. Ma, B. Yu, and M. D. F. Wong, "BOOM-explorer: RISC-V BOOM microarchitecture design space exploration," *ACM Trans. Design Autom. Electron. Syst.*, vol. 29, no. 1, pp. 1–23, 2023.
- [32] S. Daulton, M. Balandat, and E. Bakshy, "Parallel Bayesian optimization of multiple noisy objectives with expected Hypervolume improvement," in *Proc. 35th Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2021, pp. 2187–2200.
- [33] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2016, pp. 785–794. [Online]. Available: <http://dx.doi.org/10.1145/2939672.2939785>
- [34] F. Pedregosa et al., "Scikit-learn: Machine learning in python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, Nov. 2011.



Xuyang Zhao received the B.E. degree in microelectronics science and engineering from Fudan University, Shanghai, China, in 2019, where he is currently pursuing the Ph.D. degree in microelectronics and solid-state electronics.

His research interests lie in the areas of Bayesian optimization, high-dimensional optimization, multiobjective optimization, and RISC-V microarchitecture design space exploration.



Fan Yang (Member, IEEE) received the B.S. degree from Xi'an Jiaotong University, Xi'an, China, in 2003, and the Ph.D. degree from Fudan University, Shanghai, China, in 2008.

He is currently a Full Professor with the School of Microelectronics, Fudan University. His research interests include model order reduction, circuit simulation, high-level synthesis, acceleration of artificial neural networks, and yield analysis and design for manufacturability.



Tianning Gao received the B.S. degree in information engineering from the Beijing University of Posts and Telecommunications, Beijing, China, in 2015, and the M.S. degree in electrical engineering from The University of Texas at Dallas, Richardson, TX, USA, in 2017, where he is currently pursuing the Ph.D. degree in electrical engineering.

His current research includes RISC-V processor optimization and ASIC designs.



Sheng-Guo Wang (Life Senior Member, IEEE) received the B.S. degree from the University of Science and Technology of China, Beijing, China, in 1967, the M.S. degrees in electrical engineering from the University of Science and Technology of China, Hefei, China, in 1981, and the Ph.D. degree in electrical and computer engineering from the University of Houston, Houston, TX, USA, in 1994.

He is currently a Professor with The University of North Carolina at Charlotte, Charlotte, NC, USA. His current research interests include systems, circuits, control, modeling, robust control, structural control, intelligent control, GPS, communications, computer networks, mathematical, and numerical analysis, algorithms, and applications.



Zheng Wu received the B.E. degree in integrated circuit design and integrated system from the Huazhong University of Science and Technology, Wuhan, Hubei, China, in 2020. He is currently pursuing the Ph.D. degree with the State Key Lab of Integrated Chips and System, Microelectronics Department, Fudan University, Shanghai, China.

His current research interests include processor architecture design space exploration and automated deep learning compilation.



Dian Zhou (Senior Member, IEEE) received the B.S. degree in physics and the M.S. degree in electrical engineering from Fudan University, Shanghai, China, in 1982 and 1985, respectively, and the Ph.D. degree in electrical and computer engineering from the University of Illinois, Champaign, IL, USA, in 1990.

He joined the University of North Carolina at Charlotte, Charlotte, NC, USA, as an Assistant Professor in 1990, where he became an Associate Professor in 1995. He joined The University of Texas at Dallas, Dallas, TX, USA, as a Full Professor in 1999. He is currently a visiting scholar with Fudan University, Shanghai, China. His research interests include high-speed VLSI systems, CAD tools, mixed-signal ICs, and algorithms.



Zhaori Bi (Member, IEEE) received the B.Eng. degree in electronic information engineering from the Wuhan University of Technology, Wuhan, China, in 2011, and the M.S. and Ph.D. degrees in electrical engineering and computer engineering from the University of Texas at Dallas, Richardson, TX, USA, in 2013 and 2017, respectively.

He is currently an Assistant Professor with Fudan University, Shanghai, China. His current research interests include mixed-signal system-on-a-chip design, circuit performance optimization, and applications in medical AI.



Xuan Zeng (Senior Member, IEEE) received the B.S. and Ph.D. degrees in electrical engineering from Fudan University, Shanghai, China, in 1991 and 1997, respectively.

She is currently a Full Professor with the School of Microelectronics, Fudan University, where she served as the Director for the State Key Laboratory of Application Specific Integrated Circuits and Systems, Fudan University from 2008 to 2012. She was a Visiting Professor with the Department of Electrical Engineering, Texas A&M University, College Station, TX, USA, and the Microelectronics Department, Technische Universiteit Delft, Delft, The Netherlands, in 2002 and 2003, respectively. Her current research interests include analog circuit modeling and synthesis, design for manufacturability, high-speed interconnect analysis and optimization, and circuit simulation.

Prof. Zeng received the Changjiang Distinguished Professor with the Ministry of Education Department of China in 2014, the Chinese National Science Funds for Distinguished Young Scientists in 2011, the First-Class of Natural Science Prize of Shanghai in 2012, the 10th For Women in Science Award in China in 2013, and the Shanghai Municipal Natural Science Peony Award in 2014. She received the Best Paper Award from the 8th IEEE Annual Ubiquitous Computing, Electronics and Mobile Communication Conference 2017.



Changhao Yan (Member, IEEE) received the B.E. and M.E. degrees from the Huazhong University of Science and Technology, Wuhan, China, in 1996 and 2002, respectively, and the Ph.D. degree in computer science from Tsinghua University, Beijing, China, in 2006.

He is currently a Full Professor with the School of Microelectronics, Fudan University, Shanghai, China. His current research interests include the parasitic parameter extraction of interconnects, parallel algorithms for large scale computation, design for manufacturability, the robust analysis of circuits, and AI and machine learning in medicine.